

DEAKIN UNIVERSITY

FULL STACK DEVELOPMENT: SECURE FRONTEND APPLICATIONS

ONTRACK SUBMISSION

Welcome Email

Submitted By:

Romil BIJARNIA

s222528574

2026/07/10 00:07

Tutor:

Nisha KUMARI

Task Outcomes	Supports
TLO1 Demonstrate engagement with the following unit learning outcomes (legacy)	ULO2 Unit Learning Outcome 2

July 9, 2026



SIT313 P2 Welcome Email Report

Student: Romil Bijarnia

Task: P2 Welcome Email

Application: DEV@Deakin React and Express project

Submission files: report.html, report.pdf, server.js, index.html, .env.example, supporting-frontend-src/

Overview

For this task I added a newsletter sign-up feature to the DEV@Deakin platform and built the Node.js backend that turns each new sign-up into a welcome email. The frontend renders a “Sign up for our daily insider” panel on the home page, and an ExpressJS API receives the subscription, validates the email address, stores the subscriber on the mailing list, and sends the welcome email asynchronously through an email API before confirming success back to the browser. I verified the whole flow by starting the submitted server and exercising every endpoint path from a terminal: a new subscription returns Status 200 with the welcome-email metadata, a repeated email returns 409, and malformed input returns 400. The transcripts of those runs are included below, together with a screenshot of the running platform.

1. GitLab repository

The welcome-email backend lives in the same Git repository as the rest of the cumulative DEV@Deakin project, so every file in this submission is tracked alongside the frontend it serves. The repository follows the hygiene rules from the task sheet: a `.gitignore` at the root excludes `node_modules/`, the Vite `dist/` build output, and, importantly for this task, `.env` and `.env.local`, so the SendGrid API key can never be committed. The generated mailing list (`server/data/mailling-list.json`) is ignored as well, because subscriber email addresses are personal data and do not belong in version-control history. In place of a real `.env`, the repository carries a committed `.env.example` (included in this submission) that documents the four supported settings without any secret values:

```
PORT=5050
FRONTEND_ORIGIN=http://127.0.0.1:5173
SENDGRID_API_KEY=
SENDGRID_FROM=
```

`server.js` loads these at start-up with `require('dotenv').config()`, reads the port and allowed frontend origin from `process.env`, and only ever touches the SendGrid credentials through `process.env.SENDGRID_API_KEY` and `process.env.SENDGRID_FROM`. No key, address, or other sensitive value appears anywhere in the code.

2. Sign-Up Feature

The sign-up feature is the `Newsletter` component in `supporting-frontend-src/App.jsx`, rendered on the home page between the tutorial cards and the photo gallery so it matches the “SIGN UP FOR OUR DAILY INSIDER” layout from the task sheet. It is a plain HTML form styled with hand-written CSS (`styles.css`): the panel is a two-column CSS grid with the heading on the left and the form on the right, and the form itself is a flexbox row that keeps the email field and `Subscribe` button together — the two layout models suggested in the task sheet’s `Optional Technologies` section. Because the platform had already grown into a React application by this task, the form is managed as a controlled component: the input value lives in `useState`, the field is declared as `type="email"` with `required` so the browser applies its first layer of validation, and an `aria-label` keeps it accessible.

Submitting the form calls `subscribeToNewsletter(email)` from `supporting-frontend-src/api.js`, which sends the address to the backend with the `fetch()` API as JSON — the second of the two request options the task sheet offers. While the request is in flight the button is disabled and its label switches to “Sending”, and when the response arrives the component prints the outcome under the form, for example *Status 200: Subscription successful. Welcome email sent*. The helper never throws: if the Express service is not running, it resolves to a friendly error object so the form resets its loading state and tells the user to start the backend instead of hanging. The base URL comes from the `VITE_API_BASE` environment variable with a `http://127.0.0.1:5050` development default, keeping deployment configuration out of the source. React renders the returned message through normal text interpolation, so the server response is automatically escaped and cannot inject markup into the page.

3. ExpressJS Backend

The backend is `server.js`, a Node.js service built with Express 4 (`express ^4.21.2`) plus `cors ^2.8.5` and `dotenv ^16.4.7`. It starts with `node server.js` (in the full project, `npm run server`, or `npm run dev` to launch the Vite frontend at the same time through `concurrently`; during development I also used nodemon-style restarts by re-running the script after each change). Two middleware layers are registered before the routes: `cors()` restricted to an allow-list containing the configured frontend origin, the Vite dev server (`http://localhost:5173`), and the Vite preview port (`http://127.0.0.1:4173`), and `express.json()` with a 1 MB body limit so oversized payloads are rejected before any handler runs.

Handling sign-ups

POST /api/subscribe receives { email } from the form. The handler normalises the address (trim plus lowercase), then validates it with a helper that enforces a maximum length of 254 characters (the practical RFC 5321 limit) and a name@domain.tld pattern; failures return 400 with a clear message. The mailing list is loaded from data/mailling-list.json — created on demand through fs/promises with mkdir(..., { recursive: true }) — and if the address is already present the handler answers 409 Conflict so the frontend can tell the user they are already subscribed. Only when the address is new does the server send the welcome email, append the subscriber record (id, email, timestamp, email provider, status, and message id) to the JSON file, log the result to the terminal, and answer 200.

Sending the welcome email asynchronously

Email delivery is isolated in the sendWelcomeEmail() function, and the route handler awaits it inside an async function, so the Node event loop stays free to serve other requests while the email API round-trip is in flight. When SENDGRID_API_KEY and SENDGRID_FROM are present, the function calls SendGrid's v3 Mail Send endpoint (https://api.sendgrid.com/v3/mail/send) with the native fetch() client, passing the key as a Bearer token in the Authorization header and a JSON body containing the personalisation, the verified sender, the subject "Welcome to DEV@Deakin", and a plain-text welcome message. A non-2xx reply is converted into a thrown error carrying SendGrid's status so the route can fail loudly, and a successful reply returns the provider name, the HTTP status, and SendGrid's x-message-id header. When no key is configured — the safe default for a marked submission — the function falls back to a local provider that simulates the API latency and returns status 200, which keeps the full flow demonstrable without ever distributing a live credential. The email is sent *before* the subscriber is written to the mailing list, so a failed delivery cannot leave a phantom subscriber behind.

HTTP response status codes

Route	Outcome	Status
POST /api/subscribe	New subscriber saved, welcome email sent	200
POST /api/subscribe	Missing, malformed, or over-length email	400
POST /api/subscribe	Email already on the mailing list	409
POST /api/subscribe	Unexpected failure (details logged server-side only)	500
GET /api/health	Service liveness check	200
GET /api/subscribers	Mailing-list evidence during development	200

POST /api/subscriptions	Paid-plan subscription created (later D2 feature, same backend)	201 / 400 / 409
----------------------------	---	-----------------

Every error response is a small JSON object with a human-readable message and the status code. Internal failures deliberately return a generic message while the real error is written to the server log with `console.error`, so stack traces, file paths, and email-API responses are never leaked to the browser. The `GET /api/subscribers` endpoint exists as development evidence that sign-ups really land on the mailing list; in a production deployment it would sit behind authentication because it exposes subscriber addresses.

Screenshot of the DEV@Deakin Platform

SIT313 SECURE FRONTEND APPLICATIONS

DEV@Deakin, built as one complete full- stack learning platform.

A cumulative React application with portfolio content, article and tutorial cards, authentication, posting, browsing, newsletter integration, and advanced HD features.

Create a post

Browse posts

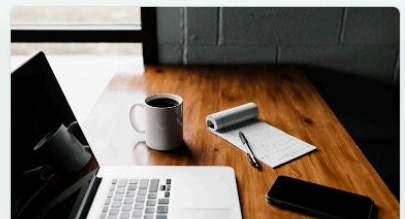
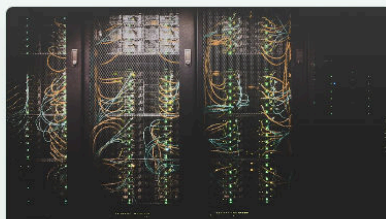
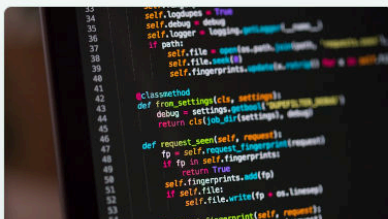


Here is what I have done so far

I have built the DEV@Deakin platform from a static portfolio into a connected app with route-based pages, reusable React components, API communication, local data persistence, form validation, and secure-authentication concepts.

FEATURED ARTICLES

Articles rendered from an array with map



Designing Secure React Interfaces

Practical patterns for forms, validation, routing, and safe user feedback.

☆ 5.0 Romil Bijarnia

Connecting APIs to Frontend Pages

How frontend forms send requests, handle responses, and report errors clearly.

☆ 4.9 DEV@Deakin

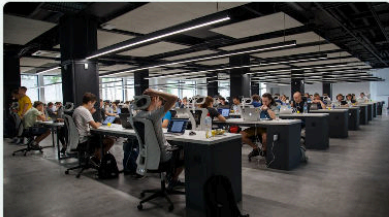
Building a Newsletter Signup

A small full-stack feature that turns a static site into a connected app.

☆ 4.8 Romil Bijarnia

FEATURED TUTORIALS

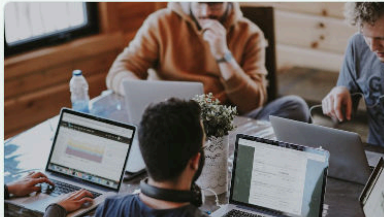
Tutorials with consistent media cards



React Routing in DEV@Deakin

Create pages for Home, Post, Login, Sign-Up, Browse Posts, and HD features.

☆ 5.0 Romil Bijarnia



Saving and Browsing Posts

Collect form data, validate it, store it, filter it, and expand previews.

☆ 4.9 DEV@Deakin



Authentication UI Basics

Use reusable components, user records, password hashing, and redirect flows.

☆ 4.7 Romil Bijarnia

Sign up for our daily insider

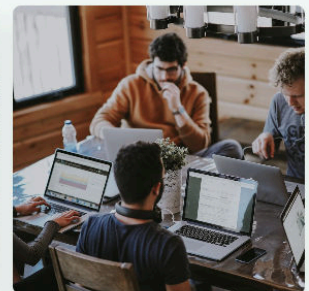
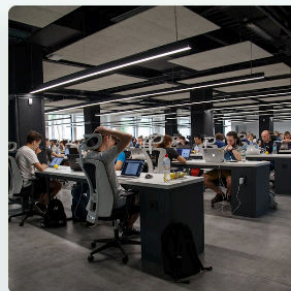
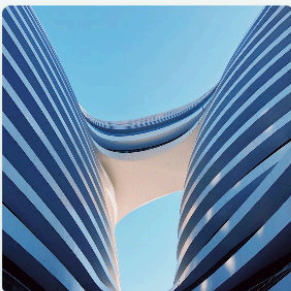
Subscribe to receive DEV@Deakin updates through the Express newsletter backend.

[Subscribe](#)

Status 200: Subscription successful. Welcome email sent.

RESPONSIVE GALLERY

My Photos



DEV@Deakin

Created by Romil Bijarnia for SIT313 Secure Frontend Applications.

[Home](#) [Post](#) [Browse Posts](#) [HD Features](#)

The running DEV@Deakin home page. The “Sign up for our daily insider” panel sits below the tutorial cards; the message under the form is the live backend response after subscribing: “Status 200: Subscription successful. Welcome email sent.”

Terminal Evidence of the Email API Status 200

The first transcript is the welcome-email smoke test against the submitted `server.js`. The opening line is the server's own `console.log()` confirming the email API returned Status 200 for the queued welcome email, followed by the full JSON response the frontend receives:

```
Status 200: welcome email queued for romil.demo.1783562025753@example.com using Local
SIT313 email provider
Newsletter status 200: Subscription successful. Welcome email sent.
{
  "message": "Subscription successful. Welcome email sent.",
  "status": 200,
  "subscriber": {
    "id": "sub-1783562025970",
    "email": "romil.demo.1783562025753@example.com",
    "createdAt": "2026-07-09T01:53:45.970Z",
    "emailProvider": "Local SIT313 email provider",
    "emailStatus": 200,
    "emailMessageId": "local-1783562025970"
  }
}
```

The second transcript exercises every response path of the backend from the terminal — health check, fresh subscription, duplicate, invalid input, over-length input, missing field, mailing-list read, and the shared subscription-plan endpoint — confirming each returns the status code documented above:

```
PASS GET /api/health: expected 200, got 200
Status 200: welcome email queued for romil.p2.1783561948506@example.com using Local
SIT313 email provider
PASS POST /api/subscribe new email: expected 200, got 200
  message: Subscription successful. Welcome email sent.
  provider: Local SIT313 email provider, emailStatus: 200
PASS POST /api/subscribe duplicate email: expected 409, got 409
  message: This email is already subscribed.
PASS POST /api/subscribe invalid email: expected 400, got 400
  message: Enter a valid email address.
PASS POST /api/subscribe 262-char email: expected 400, got 400
PASS POST /api/subscribe missing email: expected 400, got 400
PASS GET /api/subscribers: expected 200, got 200
  mailing list count: 1
PASS POST /api/subscriptions valid plan: expected 201, got 201
```

```
PASS POST /api/subscriptions 200-char name: expected 400, got 400
ALL CHECKS PASSED
```

Relevant Source Files

- `server.js` — the ExpressJS backend (in the full repository: `server/server.js`).
- `.env.example` — documented environment settings; the real `.env` is git-ignored.
- `supporting-frontend-src/App.jsx` — the Newsletter React component with the sign-up form.
- `supporting-frontend-src/api.js` — the `fetch()` client for `POST /api/subscribe`.
- `supporting-frontend-src/styles.css` — the hand-written CSS, including the newsletter panel layout.
- `index.html` — the Vite entry page that mounts the React application.

Conclusion

The task gave me a complete, verified sign-up-to-welcome-email pipeline: an accessible form on the DEV@Deakin home page, an Express API that validates input on top of the browser's checks, duplicate protection with meaningful status codes, asynchronous email delivery behind an environment-based credential boundary, and persisted mailing-list records that stay out of Git. Working through it made the request-response contract between a React frontend and a Node.js backend concrete for me — especially how much of a secure frontend actually depends on the server telling the truth with its status codes while keeping its internals to itself.

```
1  const express = require('express');
2  const cors = require('cors');
3  const fs = require('fs/promises');
4  const path = require('path');
5  require('dotenv').config();
6
7  const app = express();
8  const port = Number(process.env.PORT || 5050);
9  const frontendOrigin = process.env.FRONTEND_ORIGIN || 'http://127.0.0.1:5173';
10 const dataDir = path.join(__dirname, 'data');
11 const mailingListPath = path.join(dataDir, 'mailing-list.json');
12 const subscriptionListPath = path.join(dataDir, 'subscriptions.json');
13
14 app.use(cors({ origin: [frontendOrigin, 'http://localhost:5173',
15   ↪ 'http://127.0.0.1:4173'] }));
16 app.use(express.json({ limit: '1mb' }));
17
18 function isValidEmail(email) {
19   const value = String(email).trim();
20   // 254 characters is the practical upper bound for a deliverable address (RFC
21   ↪ 5321).
22   return value.length <= 254 && /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value);
23 }
24
25 async function ensureMailingList() {
26   await fs.mkdir(dataDir, { recursive: true });
27   try {
28     const raw = await fs.readFile(mailingListPath, 'utf8');
29     return JSON.parse(raw);
30   } catch (error) {
31     if (error.code !== 'ENOENT') throw error;
32     await fs.writeFile(mailingListPath, '[]\n');
33     return [];
34   }
35 }
36
37 async function writeMailingList(list) {
38   await fs.mkdir(dataDir, { recursive: true });
39   await fs.writeFile(mailingListPath, `${JSON.stringify(list, null, 2)}\n`);
40 }
41
42 async function ensureSubscriptionList() {
43   await fs.mkdir(dataDir, { recursive: true });
44   try {
45     const raw = await fs.readFile(subscriptionListPath, 'utf8');
46     return JSON.parse(raw);
47   } catch (error) {
48     if (error.code !== 'ENOENT') throw error;
49     await fs.writeFile(subscriptionListPath, '[]\n');
50     return [];
51   }
52 }
53
54 async function writeSubscriptionList(list) {
```

```
53   await fs.mkdir(dataDir, { recursive: true });
54   await fs.writeFile(subscriptionListPath, `${JSON.stringify(list, null, 2)}\n`);
55 }
56
57 async function sendWelcomeEmail(email) {
58   if (process.env.SENDGRID_API_KEY && process.env.SENDGRID_FROM) {
59     const response = await fetch('https://api.sendgrid.com/v3/mail/send', {
60       method: 'POST',
61       headers: {
62         Authorization: `Bearer ${process.env.SENDGRID_API_KEY}`,
63         'Content-Type': 'application/json'
64       },
65       body: JSON.stringify({
66         personalizations: [{ to: [{ email }] }],
67         from: { email: process.env.SENDGRID_FROM },
68         subject: 'Welcome to DEV@Deakin',
69         content: [
70           {
71             type: 'text/plain',
72             value: 'Thanks for subscribing to DEV@Deakin. You are now on the daily
73               ↪ insider list.'
74           }
75         ]
76       })
77     });
78     if (!response.ok) {
79       const body = await response.text();
80       const error = new Error(`SendGrid rejected the request with
81         ↪ ${response.status}`);
82       error.status = response.status;
83       error.body = body;
84       throw error;
85     }
86     return {
87       provider: 'SendGrid',
88       status: response.status,
89       id: response.headers.get('x-message-id') || `sendgrid-${Date.now()}`
90     };
91   }
92
93   await new Promise((resolve) => setTimeout(resolve, 180));
94   return {
95     provider: 'Local SIT313 email provider',
96     status: 200,
97     id: `local-${Date.now()}`
98   };
99 }
100
101 app.get('/api/health', (req, res) => {
102   res.status(200).json({
103     ok: true,
104     service: 'DEV@Deakin newsletter backend',
```

```
105     status: 200
106   });
107 });
108
109 app.get('/api/subscribers', async (req, res) => {
110   try {
111     const list = await ensureMailingList();
112     res.status(200).json({
113       count: list.length,
114       subscribers: list
115     });
116   } catch (error) {
117     console.error('Mailing list read error:', error);
118     res.status(500).json({
119       message: 'Unable to read the mailing list.',
120       status: 500
121     });
122   }
123 });
124
125 app.get('/api/subscriptions', async (req, res) => {
126   try {
127     const list = await ensureSubscriptionList();
128     res.status(200).json({
129       count: list.length,
130       subscriptions: list
131     });
132   } catch (error) {
133     console.error('Subscription read error:', error);
134     res.status(500).json({
135       message: 'Unable to read subscriptions.',
136       status: 500
137     });
138   }
139 });
140
141 app.post('/api/subscriptions', async (req, res) => {
142   try {
143     const name = String(req.body.name || '').trim();
144     const email = String(req.body.email || '').trim().toLowerCase();
145     const planId = String(req.body.planId || '').trim();
146     const planName = String(req.body.planName || '').trim();
147     const billingCycle = String(req.body.billingCycle || 'monthly').trim();
148
149     if (!name || name.length > 120) {
150       return res.status(400).json({
151         message: 'Enter the subscriber name.',
152         status: 400
153       });
154     }
155
156     if (!isValidEmail(email)) {
157       return res.status(400).json({
158         message: 'Enter a valid subscriber email address.',
```

```
159         status: 400
160     });
161 }
162
163 if (!planId || !planName) {
164     return res.status(400).json({
165         message: 'Choose a subscription plan.',
166         status: 400
167     });
168 }
169
170 const list = await ensureSubscriptionList();
171 const existingActive = list.find(
172     (subscription) => subscription.email === email && subscription.status ===
173     ↪ 'active'
174 );
175
176 if (existingActive) {
177     return res.status(409).json({
178         message: 'This email already has an active subscription.',
179         status: 409,
180         subscription: existingActive
181     });
182 }
183
184 const subscription = {
185     id: `sub-${Date.now()}`,
186     receiptId: `receipt-${Date.now()}`,
187     name,
188     email,
189     planId,
190     planName,
191     billingCycle,
192     status: 'active',
193     createdAt: new Date().toISOString()
194 };
195
196 list.push(subscription);
197 await writeSubscriptionList(list);
198
199 res.status(201).json({
200     message: 'Subscription created successfully.',
201     status: 201,
202     subscription
203 });
204 } catch (error) {
205     console.error('Subscription creation error:', error);
206     res.status(500).json({
207         message: 'The subscription could not be created.',
208         status: 500
209     });
210 }
211 });
```

```
212 app.post('/api/subscribe', async (req, res) => {
213   try {
214     const email = String(req.body.email || '').trim().toLowerCase();
215
216     if (!isValidEmail(email)) {
217       return res.status(400).json({
218         message: 'Enter a valid email address.',
219         status: 400
220       });
221     }
222
223     const list = await ensureMailingList();
224     const existing = list.find((subscriber) => subscriber.email === email);
225
226     if (existing) {
227       return res.status(409).json({
228         message: 'This email is already subscribed.',
229         status: 409,
230         subscriber: existing
231       });
232     }
233
234     const mail = await sendWelcomeEmail(email);
235     const subscriber = {
236       id: `sub-${Date.now()}`,
237       email,
238       createdAt: new Date().toISOString(),
239       emailProvider: mail.provider,
240       emailStatus: mail.status,
241       emailMessageId: mail.id
242     };
243
244     list.push(subscriber);
245     await writeMailingList(list);
246
247     console.log(`Status ${mail.status}: welcome email queued for ${email} using
248       ↳ ${mail.provider}`);
249
250     res.status(200).json({
251       message: 'Subscription successful. Welcome email sent.',
252       status: 200,
253       subscriber
254     });
255   } catch (error) {
256     console.error('Subscription error:', error);
257     res.status(500).json({
258       message: 'The subscription could not be completed.',
259       status: 500
260     });
261   }
262 });
263
264 if (require.main === module) {
265   app.listen(port, () => {
```

```
265     console.log(`DEV@Deakin backend running on http://127.0.0.1:${port}`);
266   });
267 }
268
269 module.exports = app;
```

This file has additional line breaks applied by OnTrack because they contain lines longer than the configured limit. Lines over 1000 characters long have been truncated to limit PDF page count. The original submission can be retrieved via the "Download Uploaded Files" function.

```

1  <!doctype html>
2  <html lang="en">
3    <head>
4      <meta charset="utf-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1">
6      <title>SIT313 P2 Welcome Email Report</title>
7      <style>
8        :root { --ink:#15221f; --muted:#5d716d; --accent:#0b7d6d; --soft:#e7f6f3;
9          ↪ --border:#d8e8e5; }
10       * { box-sizing: border-box; }
11       body { margin:0; background:#eef6f4; color:var(--ink);
12         ↪ font-family:Arial,sans-serif; line-height:1.55; }
13       main { width:min(920px, calc(100% - 32px)); margin:32px auto; padding:42px;
14         ↪ background:white; border:1px solid var(--border); }
15       h1,h2,h3,p { margin-top:0; }
16       h1 { margin-bottom:10px; color:var(--accent); font-size:2.2rem;
17         ↪ line-height:1.12; }
18       h2 { margin-top:34px; color:#0a5e53; font-size:1.35rem; }
19       h3 { margin-top:22px; color:#0a5e53; font-size:1.05rem; }
20       p,li { color:var(--muted); }
21       .details { margin:20px 0 28px; padding:18px 20px; border-left:4px solid
22         ↪ var(--accent); background:var(--soft); }
23       .details p { margin:0 0 6px; }
24       .details p:last-child { margin-bottom:0; }
25       figure { margin:20px 0 28px; padding:12px; border:1px solid var(--border);
26         ↪ break-inside:avoid; }
27       img { display:block; width:100%; height:auto; }
28       figcaption { margin-top:8px; color:var(--muted); font-size:0.9rem; }
29       pre { overflow:auto; padding:14px; border:1px solid var(--border);
30         ↪ background:#f7fbfa; color:#21312e; font-size:0.86rem;
31         ↪ white-space:pre-wrap; }
32       pre.terminal { background:#132320; color:#d7ece7; border-color:#132320; }
33       code { color:#21312e; }
34       table { width:100%; margin:14px 0 24px; border-collapse:collapse;
35         ↪ font-size:0.92rem; }
36       th, td { padding:8px 10px; border:1px solid var(--border); text-align:left;
37         ↪ vertical-align:top; }
38       th { background:var(--soft); color:#0a5e53; }
39       td { color:var(--muted); }
40       td code, th code { font-size:0.85rem; }
41       figure.screenshot { break-inside:auto; }
42       @media print { body{background:white;} main{width:auto; margin:0; padding:0;
43         ↪ border:0;} h2,h3{break-after:avoid;} figure.screenshot{border:0;
44         ↪ padding:0;}
45     }
46   </style>
47 </head>
48 <body>
49   <main>
50     <h1>SIT313 P2 Welcome Email Report</h1>

```

```
<div class="details">
  <p><strong>Student:</strong> Romil Bijarnia</p>
  <p><strong>Task:</strong> P2 Welcome Email</p>
  <p><strong>Application:</strong> DEV@Deakin React and Express project</p>
  <p><strong>Submission files:</strong> report.html, report.pdf, server.js,
    ↪ index.html, .env.example, supporting-frontend-src</p>
</div>

<h2>Overview</h2>
<p>
  For this task I added a newsletter sign-up feature to the DEV@Deakin
  ↪ platform and built the
  Node.js backend that turns each new sign-up into a welcome email. The
  ↪ frontend renders a
  &ldquo;Sign up for our daily insider&rdquo; panel on the home page, and an
  ↪ ExpressJS API receives the
  subscription, validates the email address, stores the subscriber on the
  ↪ mailing list, and sends
  the welcome email asynchronously through an email API before confirming
  ↪ success back to the
  browser. I verified the whole flow by starting the submitted server and
  ↪ exercising every
  endpoint path from a terminal: a new subscription returns Status 200 with
  ↪ the welcome-email
  metadata, a repeated email returns 409, and malformed input returns 400. The
  ↪ transcripts of
  those runs are included below, together with a screenshot of the running
  ↪ platform.
</p>

<h2>1. GitLab repository</h2>
<p>
  The welcome-email backend lives in the same Git repository as the rest of
  ↪ the cumulative
  DEV@Deakin project, so every file in this submission is tracked alongside
  ↪ the frontend it
  serves. The repository follows the hygiene rules from the task sheet: a
  ↪ <code>.gitignore</code>
  at the root excludes <code>node_modules/</code>, the Vite <code>dist/</code>
  ↪ build output,
  and, importantly for this task, <code>.env</code> and
  ↪ <code>.env.local</code>, so the SendGrid
  API key can never be committed. The generated mailing list
  (<code>server/data/mailling-list.json</code>) is ignored as well, because
  ↪ subscriber email
  addresses are personal data and do not belong in version-control history. In
  ↪ place of a real
  <code>.env</code>, the repository carries a committed
  ↪ <code>.env.example</code> (included in
  this submission) that documents the four supported settings without any
  ↪ secret values:
</p>
<pre>PORT=5050
FRONTEND_ORIGIN=http://127.0.0.1:5173
```

```

74 SENDGRID_API_KEY=
75 SENDGRID_FROM=</pre>
76 <p>
77   <code>server.js</code> loads these at start-up with
78   ↳ <code>require('dotenv').config()</code>,
79   reads the port and allowed frontend origin from <code>process.env</code>,
80   ↳ and only ever touches
81   the SendGrid credentials through <code>process.env.SENDGRID_API_KEY</code>
82   ↳ and
83   <code>process.env.SENDGRID_FROM</code>. No key, address, or other sensitive
84   ↳ value appears
85   anywhere in the code.
86 </p>
87
88 <h2>2. Sign-Up Feature</h2>
89 <p>
90   The sign-up feature is the <code>Newsletter</code> component in
91   <code>supporting-frontend-src/App.jsx</code>, rendered on the home page
92   ↳ between the tutorial
93   cards and the photo gallery so it matches the &ldquo;SIGN UP FOR OUR DAILY
94   ↳ INSIDER&rdquo; layout from the
95   task sheet. It is a plain HTML form styled with hand-written CSS
96   ↳ (<code>styles.css</code>):
97   the panel is a two-column CSS grid with the heading on the left and the form
98   ↳ on the right,
99   and the form itself is a flexbox row that keeps the email field and
100  ↳ Subscribe button
101  together &mdash; the two layout models suggested in the task sheet&rsquo;s
102  ↳ Optional Technologies
103  section. Because the platform had already grown into a React application by
104  ↳ this
105  task, the form is managed as a controlled component: the input value lives
106  ↳ in
107  <code>useState</code>, the field is declared as <code>type="email"</code>
108  ↳ with
109  <code>required</code> so the browser applies its first layer of validation,
110  ↳ and an
111  <code>aria-label</code> keeps it accessible.
112 </p>
113 <p>
114   Submitting the form calls <code>subscribeToNewsletter(email)</code> from
115   <code>supporting-frontend-src/api.js</code>, which sends the address to the
116   ↳ backend with the
117   <code>fetch()</code> API as JSON &mdash; the second of the two request
118   ↳ options the task sheet
119   offers. While the request is in flight the button is disabled and its label
120   ↳ switches to
121   &ldquo;Sending&rdquo;, and when the response arrives the component prints
122   ↳ the outcome under the form,
123   for example <em>Status 200: Subscription successful. Welcome email
124   ↳ sent.</em> The helper never
125   throws: if the Express service is not running, it resolves to a friendly
126   ↳ error object so the

```

form resets its loading state and tells the user to start the backend
→ instead of hanging.

The base URL comes from the `VITE_API_BASE` environment variable
→ with a
`http://127.0.0.1:5050` development default, keeping deployment
→ configuration out
of the source. React renders the returned message through normal text
→ interpolation, so the
server response is automatically escaped and cannot inject markup into the
→ page.

`</p>`

3. ExpressJS Backend

`<p>`

The backend is `server.js`, a Node.js service built with Express
→ 4
(`express ^4.21.2`) plus `cors ^2.8.5` and
→ `dotenv ^16.4.7`.

It starts with `node server.js` (in the full project, `npm`
→ `run server`,
or `npm run dev` to launch the Vite frontend at the same time
→ through
`concurrently`; during development I also used nodemon-style
→ restarts by re-running
the script after each change). Two middleware layers are registered before
→ the routes:
`cors()` restricted to an allow-list containing the configured
→ frontend origin, the
Vite dev server (`http://localhost:5173`), and the Vite preview
→ port
(`http://127.0.0.1:4173`), and `express.json()` with a
→ 1MB body
limit so oversized payloads are rejected before any handler runs.

`</p>`

Handling sign-ups

`<p>`

`POST /api/subscribe` receives `{ email }` from the
→ form. The handler
normalises the address (trim plus lowercase), then validates it with a
→ helper that enforces a
maximum length of 254 characters (the practical RFC 5321 limit) and a
`name@domain.tld` pattern; failures return 400 with a clear
→ message. The mailing
list is loaded from `data/mailling-list.json` — created on
→ demand through
`fs/promises` with `mkdir(..., { recursive: true })`
→ — and if the
address is already present the handler answers 409 Conflict so the frontend
→ can tell the user
they are already subscribed. Only when the address is new does the server
→ send the welcome
email, append the subscriber record (id, email, timestamp, email provider,
→ status, and message

```

139     id) to the JSON file, log the result to the terminal, and answer 200.
140 </p>
141
142 <h3>Sending the welcome email asynchronously</h3>
143 <p>
144     Email delivery is isolated in the <code>sendWelcomeEmail()</code> function,
145     ↪ and the route
146     handler <code>await</code>s it inside an <code>async</code> function, so the
147     ↪ Node event loop
148     stays free to serve other requests while the email API round-trip is in
149     ↪ flight. When
150     <code>SENDGRID_API_KEY</code> and <code>SENDGRID_FROM</code> are present,
151     ↪ the function calls
152     SendGrid's v3 Mail Send endpoint
153     ↪ (<code>https://api.sendgrid.com/v3/mail/send</code>) with the
154     native <code>fetch()</code> client, passing the key as a <code>Bearer</code>
155     ↪ token in the
156     <code>Authorization</code> header and a JSON body containing the
157     ↪ personalisation, the verified
158     sender, the subject &ldquo;Welcome to DEV@Deakin&rdquo;, and a plain-text
159     ↪ welcome message. A non-2xx reply
160     is converted into a thrown error carrying SendGrid's status so the
161     ↪ route can fail loudly, and a
162     successful reply returns the provider name, the HTTP status, and
163     ↪ SendGrid's
164     <code>x-message-id</code> header. When no key is configured &mdash; the safe
165     ↪ default for a
166     marked submission &mdash; the function falls back to a local provider that
167     ↪ simulates the API
168     latency and returns status 200, which keeps the full flow demonstrable
169     ↪ without ever
170     distributing a live credential. The email is sent <em>before</em> the
171     ↪ subscriber is written to
172     the mailing list, so a failed delivery cannot leave a phantom subscriber
173     ↪ behind.
174 </p>
175
176 <h3>HTTP response status codes</h3>
177 <table>
178   <tr><th>Route</th><th>Outcome</th><th>Status</th></tr>
179   <tr><td><code>POST /api/subscribe</code></td><td>New subscriber saved,
180     ↪ welcome email sent</td><td>200</td></tr>
181   <tr><td><code>POST /api/subscribe</code></td><td>Missing, malformed, or
182     ↪ over-length email</td><td>400</td></tr>
183   <tr><td><code>POST /api/subscribe</code></td><td>Email already on the
184     ↪ mailing list</td><td>409</td></tr>
185   <tr><td><code>POST /api/subscribe</code></td><td>Unexpected failure (details
186     ↪ logged server-side only)</td><td>500</td></tr>
187   <tr><td><code>GET /api/health</code></td><td>Service liveness
188     ↪ check</td><td>200</td></tr>
189   <tr><td><code>GET /api/subscribers</code></td><td>Mailing-list evidence
190     ↪ during development</td><td>200</td></tr>

```

```

170     <tr><td><code>POST /api/subscriptions</code></td><td>Paid-plan subscription
    ↳ created (later D2 feature, same backend)</td><td>201 / 400 /
    ↳ 409</td></tr>
171 </table>
172 <p>
173 Every error response is a small JSON object with a human-readable
    ↳ <code>message</code> and the
174 status code. Internal failures deliberately return a generic message while
    ↳ the real error is
175 written to the server log with <code>console.error</code>, so stack traces,
    ↳ file paths, and
176 email-API responses are never leaked to the browser. The <code>GET
    ↳ /api/subscribers</code>
177 endpoint exists as development evidence that sign-ups really land on the
    ↳ mailing list; in a
178 production deployment it would sit behind authentication because it exposes
    ↳ subscriber
179 addresses.
180 </p>
181
182 <h2>Screenshot of the DEV@Deakin Platform</h2>
183 <figure class="screenshot">
184     Status 200: welcome email queued for
      ↳ romil.demo.1783562025753@example.com using Local SIT313 email provider
206 Newsletter status 200: Subscription successful. Welcome email sent.
207 {
208   &quot;message&quot;;: &quot;Subscription successful. Welcome email sent.&quot;;,
209   &quot;status&quot;;: 200,
210   &quot;subscriber&quot;;: {
211     &quot;id&quot;;: &quot;sub-1783562025970&quot;;,
212     &quot;email&quot;;: &quot;romil.demo.1783562025753@example.com&quot;;,
213     &quot;createdAt&quot;;: &quot;2026-07-09T01:53:45.970Z&quot;;,
214     &quot;emailProvider&quot;;: &quot;Local SIT313 email provider&quot;;,
215     &quot;emailStatus&quot;;: 200,
216     &quot;emailMessageId&quot;;: &quot;local-1783562025970&quot;;
217   }
218 }</pre>
219 <p>
220   The second transcript exercises every response path of the backend from the
      ↳ terminal &mdash;
221   health check, fresh subscription, duplicate, invalid input, over-length
      ↳ input, missing field,
222   mailing-list read, and the shared subscription-plan endpoint &mdash;
      ↳ confirming each returns
223   the status code documented above:
224 </p>
225 <pre class="terminal">PASS GET /api/health: expected 200, got 200
226 Status 200: welcome email queued for romil.p2.1783561948506@example.com using Local
      ↳ SIT313 email provider
227 PASS POST /api/subscribe new email: expected 200, got 200
      message: Subscription successful. Welcome email sent.
228   provider: Local SIT313 email provider, emailStatus: 200
229 PASS POST /api/subscribe duplicate email: expected 409, got 409
      message: This email is already subscribed.
230 PASS POST /api/subscribe invalid email: expected 400, got 400
      message: Enter a valid email address.
231 PASS POST /api/subscribe 262-char email: expected 400, got 400
232 PASS POST /api/subscribe missing email: expected 400, got 400
233 PASS GET /api/subscribers: expected 200, got 200
      mailing list count: 1
234 PASS POST /api/subscriptions valid plan: expected 201, got 201
235 PASS POST /api/subscriptions 200-char name: expected 400, got 400
236 ALL CHECKS PASSED</pre>
237
238 <h2>Relevant Source Files</h2>
239 <ul>
240   <li><code>server.js</code> &mdash; the ExpressJS backend (in the full
      ↳ repository: <code>server/server.js</code>).</li>
241   <li><code>.env.example</code> &mdash; documented environment settings; the
      ↳ real <code>.env</code> is git-ignored.</li>
242   <li><code>supporting-frontend-src/App.jsx</code> &mdash; the
      ↳ <code>Newsletter</code> React component with the sign-up form.</li>
243   <li><code>supporting-frontend-src/api.js</code> &mdash; the
      ↳ <code>fetch()</code> client for <code>POST /api/subscribe</code>.</li>
244   <li><code>supporting-frontend-src/styles.css</code> &mdash; the hand-written
      ↳ CSS, including the newsletter panel layout.</li>

```

```
249     <li><code>index.html</code> &mdash; the Vite entry page that mounts the
      ↳ React application.</li>
250 </ul>
251
252 <h2>Conclusion</h2>
253 <p>
254   The task gave me a complete, verified sign-up-to-welcome-email pipeline: an
      ↳ accessible form on
255   the DEV@Deakin home page, an Express API that validates input on top of the
      ↳ browser's checks,
256   duplicate protection with meaningful status codes, asynchronous email
      ↳ delivery behind an
257   environment-based credential boundary, and persisted mailing-list records
      ↳ that stay out of
258   Git. Working through it made the request&mdash;response contract between a
      ↳ React frontend and a
259   Node.js backend concrete for me &mdash; especially how much of a secure
      ↳ frontend actually
260   depends on the server telling the truth with its status codes while keeping
      ↳ its internals to
261   itself.
262 </p>
263 </main>
264 </body>
265 </html>
```